SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0615	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO3: Experiments (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Soundariya M	**Reg. No.:**	192524342
Section / Batch:	Btech-AI&DS	**Date:**	18:08:2026

Instructions to Students

- Write algorithm and execute code both experiments.
- Each questions 50 Marks. Total: 100 Marks.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Description	Questions	Total Marks
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1 – Q2	100

SECTION A – ExperimentS

CO3-AT1-Experiments

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 ✓
Divide and Conquer
50 marks

Q2 ✓
Divide and Conquer
50 marks

2/2 answered

Q1 Divide and Conquer — 50 marks

Analyze the impact of recursion overhead in Quick Sort by measuring function call counts and stack depth. Record execution time for large datasets. Interpret how recursion contributes to total execution cost. Justify optimization techniques such as tail recursion.

Your Code

```python
import sys
import time
call_count = 0
current_depth = 0
max_depth = 0
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
def quick_sort(arr, low, high):
    global call_count, current_depth, max_depth
    call_count += 1
    current_depth += 1
    max_depth = max(max_depth, current_depth)
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
    current_depth -= 1
# Input
n = int(input())
arr = list(map(int, input().split()))
start = time.perf_counter()
quick_sort(arr, 0, n - 1)
end = time.perf_counter()
print("Sorted Array:", *arr)
print("Function Calls:", call_count)
print("Maximum Stack Depth:", max_depth)
print("Execution Time:", round((end - start) * 1000, 4), "ms")
```

Input

```
8
34 12 56 7 89 23 45 10
```

Output

```
Sorted Array: 7 10 12 23 34 45
56 89
Function Calls: 11
Maximum Stack Depth: 6
Execution Time: 0.0254 ms
```

Submitted

2/2 answered

Your Code

```python
import time
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left = []
    right = []
    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)
    return quick_sort(left) + [pivot] + quick_sort(right)
n = int(input())
data = list(map(int, input().split()))
noisy_data = data.copy()
for i in range(0, len(noisy_data), 3):
    noisy_data[i] = noisy_data[i] + 1
start = time.perf_counter()
merge_result = merge_sort(noisy_data.copy())
merge_time = (time.perf_counter() - start) * 1000
start = time.perf_counter()
quick_result = quick_sort(noisy_data.copy())
quick_time = (time.perf_counter() - start) * 1000
expected = sorted(noisy_data)
merge_correct = merge_result == expected
quick_correct = quick_result == expected
print("Original Data:", *data)
print("Noisy Data:", *noisy_data)
print("\nMerge Sort Result:", *merge_result)
print("Merge Sort Correct:", merge_correct)
print("Merge Sort Time:", round(merge_time, 4), "ms")
print("\nQuick Sort Result:", *quick_result)
print("Quick Sort Correct:", quick_correct)
print("Quick Sort Time:", round(quick_time, 4), "ms")
print("\nReliability:")
if merge_correct and quick_correct:
    print("Both algorithms correctly sorted the noisy dataset.")
else:
    print("One or more algorithms failed.")
```

Input

```
10
45 12 78 23 56 9 34 67 21 89
```

Output

```
Original Data: 45 12 78 23 56 9
34 67 21 89
Noisy Data: 46 12 78 24 56 9 35
67 21 90

Merge Sort Result: 9 12 21 24
```

Submitted

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weigh tage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Experiments	Understanding of Concepts **(25)**	Experimental Design **(30)**	Use of Tools **(20)**	Analysis of Results **(15)**	Documentation **(10)**	**Total Marks (Each - 50)**
Ex. 1						
Ex. 2						
					Overall Total (100)	**100 %**

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____